NimbusPay Ticket Triage — Fine-Tuning Report

Base model `Qwen/Qwen2.5-1.5B-Instruct` (1.54B, under the 4B cap), loaded in **16-bit** · **LoRA via PEFT** `r=16, α=32` on all attention + MLP projections → **18.46M trainable, 36.9% of the 50M cap** · **Stack** `transformers.Trainer` + `peft` + `bitsandbytes`, hand-written loss mask · Free Colab **T4**, peak **11.82 GB**, training **19.9 min**.

Headline: dev exact match 90.0%, mean field accuracy 97.9%, JSON validity 100% *(n=200), from 0.0% / 0.0% / 0.0% for the un-tuned base model on the same prompt.*

1. Audit

Method. I encoded `SCHEMA.md` as an executable validator and ran it on `dev.jsonl` *first*. Dev is stated clean, so it is a **control**: a flag there means my validator is wrong, not the data. I iterated until it emitted **zero flags on all 200 dev rows**, then pointed it at train — so every train flag is a real defect, not a misreading. It caught two bugs of mine: my repeat-contact regex fired on 16 clean rows because quoted `>` replies contain "already received a resolution"; and `phir bhi` (`dv-00118`) means "even so", not "again".

1,445 rows in → 1,350 out. 95 dropped, 191 fixed, 388 lines in `cleaning_log.csv`.

#	Defect	Rows	Action	Found by
D1	Label JSON truncated mid-string	25	**drop**	`json.loads` fails; unrecoverable
D2	Placeholder bodies (`test`, `asdf`, one empty)	10	**drop**	length floor; dev min 44 chars, these 0–13
D3	Legacy key `prio`, values `low/medium/high`	70	fix (rename+remap)	key-set histogram: two shapes
D4	`txn_date` as `DD/MM/YYYY`	30	fix	format regex; **all 30 inside D3**
D5	`txn_id` unnormalized (`np 81633 47892`)	20	fix	`NP\d{10}`; none inside D3
D6	`amount_paise` stored in **rupees**	20	fix (×100)	matches a rupee figure, not its paise value
D7	`priority` violates the spec ladder	62	fix (recompute)	rule engine, after D3/D6
D8	`needs_human` violates the spec	21	fix (recompute)	rule engine, after priority cascade
D10	Exact duplicate ticket texts	60	**drop**	text hash; **labels agree** — reweighting, not corruption

D3 mattered most. The obvious move on 70 rows keyed `prio` is to recompute priority — discarding 70 *correct* labels. The values aren't `P1/P2/P3` but `low/medium/high`, and against what the v2 ladder derives for the same row the mapping is perfect with **zero exceptions** (`low`→`P3` ×44, `medium`→`P2` ×15, `high`→`P1` ×11). `SCHEMA.md` calls itself "v2, current": this is a v1 export, so the fix is a value remap. The 30 `DD/MM/YYYY` dates falling *entirely* inside these rows confirmed one legacy exporter, not three faults.

Fix order is load-bearing. D6 must precede D7. `tr-00382` stores ₹46,094 as `46094` paise (₹460.94), pushing it under the ₹5,000 P2 threshold and making a *correct* `P2` look like a violation; recomputing priority first would have "fixed" three correct labels into wrong ones. Measured naively D7 is 135 rows; in the right order, 62. The cleaned set then passes the validator dev passes.

Rejected. *Urgency drove the over-escalation:* 33 rows are `P1` where the ladder gives `P3`, several shouting `URGENT!!`, but urgency markers appear at **24% vs 21%** of other rows — no signal, so I correct by rule and claim nothing about origin. *A bad annotator batch:* across ten id ranges of ~145, every defect class spreads evenly (priority errors 1.4%–7.2%). **Checked, nothing found:** balance/limit/fee distractors mistaken for the amount — **0 rows**, dev *and* train; fraud-worded tickets labelled otherwise — **0**; `language` corruption — none.

2. Baselines and pipeline

Identical inference contract throughout: verbatim `system+user`, Qwen chat template, greedy, `max_new_tokens=200`, `.strip()` as the only post-processing.

Run (dev, n=200)	json_valid	exact_match	mean_field_acc
B1 — base, fixed prompt	0.0%	0.0%	0.0%
B2 — base, **full rules in prompt** *(reference only)*	97.5%	0.0%	42.9%
Fine-tuned	**100.0%**	**90.0%**	**97.9%**

B2 is the informative one. Pasting the entire rulebook into the prompt fixes *format* (json_valid 0 → 97.5%) but buys almost no *task* accuracy: exact match stays at **0.0%**, and `priority` scores **0.0%** across all 200 rows while `amount_paise` reaches 81.5%. So the gap was never "the model cannot see the rules" — it extracts fields reasonably once told the schema, and still cannot apply the ladder. That is the evidence that fine-tuning, not prompting, is the right lever.

B1's 0% is real, and I nearly mis-reported it. My first run produced `json_valid=0%` with every generation being `!!!!!!!!`. That was not a model result — `!` is token id 0, and repeated token 0 is the signature of NaN logits. The cause was `attn_implementation="eager"`, which overflows fp16 on Qwen2's activation outliers. Switching to `sdpa` gave a *genuine* 0%: the model emits coherent JSON wrapped in ```` ```json ```` fences with its own invented schema (`status`, `ticket_id`, `customer_name`). Same number, completely different meaning. `load_base` now asserts the logits are finite, so that failure can never again pass as a result.

Chat template. Qwen2.5 is ChatML with a real `system` role, so the fixed prompt passes through untouched; training uses the *same* `apply_chat_template` call as inference — no train/serve skew.

Loss on the right tokens. I left TRL's `SFTTrainer` deliberately: it masks by string-matching a `response_template`, and a silent miss trains the model to reproduce tickets with a healthy-looking loss curve and no error. I build the mask directly — everything up to and including `<|im_start|>assistant\n` is `-100` — and **assert** the unmasked span decodes back to exactly the label. Verified live: supervised span `133..214`, **82 of 215 tokens**, decoding to the label and nothing else.

Sequence length, from evidence. Measured, not guessed: median **168** tokens, p90 **214**, p99 **1936**, max **2215** — bimodal, with **38 rows (2.8%)** carrying quoted `>` history. The label sits at the *end*, so right-truncation amputates the **target**; `MAX_SEQ_LEN=2240` covers the longest sequence. *Rejected:* dropping long rows — 11 of 400 test rows are long. This choice has a cost: at `bs=4` it OOM'd the T4 at 13.84/14.56 GiB, because `group_by_length` packs the long outliers into one batch. Fixed with `bs=2 × accum=8` (**effective batch unchanged at 16**) plus gradient checkpointing → peak **11.82 GB**.

Capacity. `r=16` → 18.46M. The cap binds: `r=32` is 36.9M and `r=64` **is 73.9M, breaching 50M**, so the rank sweep is 16 vs 32. LR 2e-4 cosine, 3% warmup, 3 epochs = **255 steps**; mean training loss 0.064, with the logged curve falling 0.491 (step 25) → 0.003 (step 250). fp16 only — T4 is Turing, no bf16, asserted at load.

3. Results and error analysis

Per-field (fine-tuned, dev): `amount_paise` 100% · `txn_id` 100% · `channel` 100% · `language` 100% · `txn_date` 99.0% · `category` 98.0% · `needs_human` 94.5% · `priority` **91.5%**.

Per-category: `other` / `kyc` / `account_access` **100%** · `offers` 94.4% · `refund` 89.1% · `payment_failed` 81.1% · `fraud` **80.0%**.

The three categories at 100% are exactly the three where the spec forces every transaction field to `null`. Every remaining error is in the categories where the ladder actually fires — the model has essentially finished learning *extraction* and now fails only at *applying rules to what it extracted*.

Root cause vs cascade. The spec is a dependency graph (`needs_human` ← `priority` ← `category+amount_paise`), so one upstream error surfaces as three wrong fields. Classifying each failure by root cause, and labelling any field that is correct *given the model's own upstream prediction* a cascade:

```
20/200 dev rows have at least one error
ROOT CAUSES                                          CASCADES
  9  priority: near a rupee threshold                  11  needs_human: follows its own wrong priority
  5  priority: ladder misapplied                        3  priority: correct for its own wrong inputs
  4  category: fraud confusions
  2  txn_date: relative-date arithmetic off by +1 day
total wrong fields 34 = 20 root + 14 cascade
```

Naive counting reports "`needs_human` wrong 11 times"; **zero** of those are independent — all follow a wrong priority. 34 wrong fields are **20 real mistakes**, and **9 of 20 (45%) are the ₹5,000/₹50,000 boundary**. The classifier is validated by 12 unit tests that perturb a gold label in a known way and assert the diagnosis; all 12 pass. One ordering bug surfaced that way — the generic "invented a date" bucket sat *before* the quoted-email check, hiding the specific cause.

Test set (400 rows). 400/400 parse as JSON, 0 with a wrong key set. The saved adapter was reloaded from disk and regenerated 12 random rows: **12/12 identical** to `predictions_test.jsonl`. But the test half is visibly harder: **7 rows (1.75%) contain values outside the schema's domain**, versus **zero on dev** — `priority: "PUR"`, `channel: "apple_pay"`, four `txn_id`s that are bare reference numbers, and `txn_date: "2026-07-00"` (an impossible date: the ticket says "6 din pehle" against `Received 2026-07-06`, and the model subtracted 6 from the *day digits* instead of doing date arithmetic). Post-generation repair is forbidden, so these stand — but they size the dev→test gap honestly and point at exactly the weakness below.

4. What I would do next

1. **Threshold counterfactuals (built, not yet trained).** 45% of root errors are the two rupee boundaries, which only **106 of 1,350 rows** exercise. Since the rule engine is exact on dev it works as a **label oracle**: mutate one trigger in a real ticket and recompute deterministically. Four generators produce **720 minimal pairs, all passing the validator, zero re-derivation mismatches**, taking threshold coverage 106 → 264. This is experiment E6 and is the single highest-expected-value change.
2. **Dependency-ordered output keys (built, not yet trained).** `score.py` scores via `pred[k]` lookups and `set(pred)==set(KEYS)`, so key **order is not scored** — verified against the real scorer. The schema puts `priority` at position 2 but it is a function of `amount_paise` at position 3, so the model must apply two thresholds **without writing the amount down**. The results make this concrete: `amount_paise` is **100%** while `priority` is **91.5%** — the information is present and unused. Reordering targets to `category → amount_paise → priority → needs_human → …` gives chain-of-thought inside a fixed schema at zero token cost (E5). This exploits the scorer's order-insensitivity, which I state plainly rather than leave to be discovered.
3. **Extend the oracle to** `txn_date`, the one trigger E6 does not cover — shift the `Received` header, restate the relative phrase, recompute by rule. `2026-07-00` shows the model doing digit arithmetic rather than date arithmetic.
4. **A second seed** before believing any gap; and oversampling `fraud` (6.2% of rows, weakest category at 80.0%, and it forces two other fields).

Honest gap: the Part D table is not filled in. This run used `RUN_EXPERIMENTS=False`, which produces the final model, both baselines, the error analysis and the test predictions inside one free-tier session; the reference arm E0 *is* the final run above (cleaned data, 16-bit, r=16, lr 2e-4, 3 epochs → 90.0% / 97.9% / 11.82 GB / 19.9 min). The remaining seven arms are coded and runnable with one flag, and need a further ~2.5 h of T4 time.

5. AI tools

I used **Claude (Claude Code)** throughout, as I would on the job: it wrote the validator, cleaner, probe scripts, notebook, loss-mask assertion and experiment harness, drove the Colab run through the browser, and drafted this report from real output. The *method* — encode the spec, use dev as a control, fix in dependency order — came out of the back-and-forth; the D3 finding came from reading the mismatch table it produced, not from the model asserting it. Both innovations came from asking where the headroom was: key order from checking what `score.py` is actually sensitive to, counterfactuals from counting rows per trigger.

Where I overrode it: it reached for TRL's `SFTTrainer` and I moved to a hand-written mask so masking is an assertion, not a default; its "urgency drives over-escalation" hypothesis measured out at 24% vs 21% and is reported as rejected; its first error-attribution ordering had the bug the unit tests caught. It also pinned `torch==2.4.1` (no cp313 wheel — Colab now runs Python 3.13) and chose `eager` attention, both of which failed live and were fixed on the T4.

Every number here was produced by the run in `notebook.ipynb` and is reproducible from the saved adapter.